SCALING SELF-SUPERVISED REPRESENTATION LEARNING FOR SYMBOLIC PIANO PERFORMANCE

Louis Bradshaw[1,4] **Honglu Fan**[3,4] **Alexander Spangher**[2,4] **Stella Biderman**[4] **Simon Colton**[1]
[1] Queen Mary University of London [2] University of Southern California
[3] University of Geneva [4] EleutherAI
l.b.bradshaw@qmul.ac.uk, honglu.fan@unige.ch, spangher@usc.edu

ABSTRACT

We study the capabilities of generative autoregressive transformer models trained on large amounts of symbolic solo-piano transcriptions. After first pretraining on approximately 60,000 hours of music, we use a comparatively smaller, high-quality subset, to finetune models to produce musical continuations, perform symbolic classification tasks, and produce general-purpose contrastive MIDI embeddings by adapting the SimCLR framework to symbolic music. When evaluating piano continuation coherence, our generative model outperforms leading symbolic generation techniques and remains competitive with proprietary audio generation models. On MIR classification benchmarks, frozen representations from our contrastive model achieve state-of-the-art results in linear probe experiments, while direct finetuning demonstrates the generalizability of pretrained representations, often requiring only a few hundred labeled examples to specialize to downstream tasks.

1. INTRODUCTION

Modern machine learning systems increasingly utilize *self-supervised learning* (SSL) as a core component of their training pipeline. In this paradigm, general-purpose representations are learned during an initial phase of self-guided learning, which can then be adapted to specialized tasks, often outperforming purely supervised approaches, particularly when access to supervised data is limited [1].

As in other fields, recent work using neural networks to model symbolic music has started to adopt SSL [2–5]. However, the symbolic music data that these models are trained on is typically created manually, in a labor-intensive process. Acquiring it at the scale common for other modalities (e.g., text, images, audio) is challenging. Consequently, successful research often involves training from scratch on datasets such as Lakh and IMSLP [6, 7], with research problems formulated around tasks that directly align with these datasets (e.g. multi-track symbolic music generation). This contrasts with other domains where substantial efforts have produced generalist models trained at an extreme scale, such as LLaMA and CLIP [8,9], which provide strong foundations for research in data-limited settings [10, 11]. These constraints on symbolic music research become particularly clear when considering advancements in the neighboring area of audio modeling, where large-scale models including AudioGen and AudioLM [12, 13], alongside their underlying neural audio codecs [14, 15], have driven a broad range of advancements in music generation [16–18], and where SSL has been applied at scale to develop effective, general-purpose embedding models [19, 20].

Fortunately, strong progress has been made towards alleviating data bottlenecks for symbolic music research by leveraging neural networks trained for *automatic music transcription* (AMT) [21]. In the restricted domain of solo-piano audio recordings, modern AMT models achieve highly reliable note-identification accuracy [22–24], enabling automated dataset curation pipelines that crawl raw

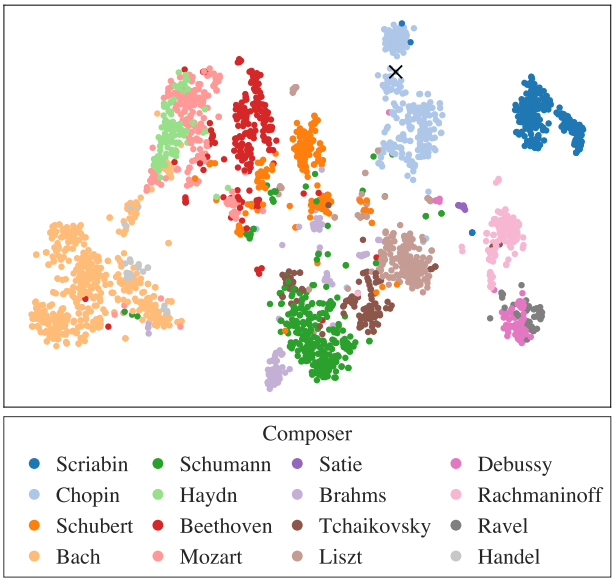


Figure 1. t-SNE visualisation of contrastive embeddings of classical compositions, trained on MIDI data without external metadata. The cross (×) highlights Chopin's *Waltz in A minor*, which was discovered [1] after the training data was compiled, ensuring that it was not included.

© L. Bradshaw, H. Fan, A. Spangher, S. Biderman and S. Colton. Licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0). **Attribution:** L. Bradshaw, H. Fan, A. Spangher, S. Biderman and S. Colton, "Scaling Self-Supervised Representation Learning for Symbolic Piano Performance", in *Proc. of the 26th Int. Society for Music Information Retrieval Conf.,* Daejeon, South Korea, 2025.

[1] See Javier C. Hernández, "Hear a Chopin Waltz Unearthed After Nearly 200 Years," *The New York Times*, Oct. 27, 2024.

audio and transcribe it into MIDI using a combination of web scraping, audio-based processing, and AMT methods [25–27]. Moreover, as this symbolic data is transcribed from real recordings, it captures the subtleties and dynamics of human performance. Recently, this combined progress has resulted in a new dataset of symbolic music, *Aria-MIDI* [28], comprising transcriptions of solo-piano recordings gathered at scale from YouTube, which has been made available for public use. At ~100k hours, Aria-MIDI is orders of magnitude larger than similar datasets [25], presenting a unique opportunity to investigate the application of scaling SSL methods to symbolic music modeling.

Building on this, in this work we leverage Aria-MIDI to pretrain a generative transformer model via next-token prediction, using it as a foundation to explore the effectiveness of SSL techniques applied to symbolic music at a scale closer to recent applications in the text, image, and audio domains. We evaluate our model across two dimensions: generative modeling and representation learning. For generative capabilities, we conduct human listening tests comparing piano continuations generated by our model, while for representation learning we measure the ability of the pretrained model to adapt to MIR classification tasks via finetuning. To explore applications to similarity and retrieval tasks, we propose and analyze a novel self-supervised adaptation of the contrastive learning framework to symbolic music, which finetunes our model to produce embeddings that capture performance and composition-level features, as demonstrated by the natural composer clustering visualized in Figure 1. In both evaluation settings, we compare against symbolic and audio-based baselines. Overall, our experiments provide strong evidence that scaling SSL is a promising approach to tackling difficult tasks across symbolic MIR. Our key contributions are the following:

1. We introduce and open-source *Aria* [2] , a pretrained autoregressive transformer model trained on transcriptions of piano recordings. Through human listening tests, we show it generates coherent continuations from short musical prompts, outperforming Anticipatory Music Transformer [29] and rivaling proprietary audio models like Suno 3.5 [30].

2. We further demonstrate the effectiveness of large-scale pretrained representations for symbolic MIR through two approaches: (1) directly finetuning our model for classification tasks, achieving strong performance when labeled examples are extremely limited, and (2) proposing a novel adaptation of contrastive learning that produces an embedding model achieving state-of-the-art accuracy in linear probe experiments including composer, genre, and style detection. Critically, we show that this contrastive approach is effective *only* when applied as a secondary finetuning phase.

In addition to our models, we release a MIDI preprocessing and tokenization library designed to scale to large datasets and, although this work focuses on solo piano, to natively support multi-track MIDI files. Together, these contributions may serve as a foundation for future research in symbolic music modeling.

[2] Available at: https://github.com/eleutherai/aria

2. RELATED WORK

Our work relates to many sub-areas of computational music, generative modeling, and representation learning. In this section, we focus on related work specific to the subfield of symbolic music modeling.

The field of symbolic music generation using neural networks has advanced rapidly. Prior to the introduction of transformers, models such as DeepBach [31] and Coconet [32] demonstrated that neural networks are effective tools for modeling musical harmonies in Baroque music. The autoregressive paradigm for symbolic music generation, which models music as a stream of *tokens*, gained traction by adapting architectures from natural language processing [33]. This approach was extended by [34] to incorporate expressive onset and duration timings, enabling generated music to more closely emulate human performance.

Music Transformer [35] was a seminal work demonstrating the power and scalability of the autoregressive approach. The authors trained a transformer decoder on the MAESTRO dataset [36], a collection of expressive MIDI piano recordings, and showed that autoregressive models could effectively learn long-term musical dependencies. Subsequent work from the same authors provided strong evidence that the musical and creative capabilities of their model scale well with dataset size [37], reinforcing the value of curating large-scale piano transcription datasets as a future direction, a central premise we explore in our work.

Building on this foundation, MuseNet [38] expanded this approach by adding multi-track support to its MIDI tokenizer and training a larger model on a diverse corpus of multi-instrument data, including MAESTRO. Alternative tokenization schemes, such as REMI [39], have also been influential. Variations of REMI have been adopted by models including Museformer [40], Figaro [41], and MuseCoco [42], which all introduced methods for conditioning generation on various musical features. Other research has explored representations beyond MIDI, such as the ABC notation [43] used by MuPT [3]. More recently, Anticipatory Music Transformer [29] was introduced as a versatile, state-of-the-art model for prompt continuation and infilling tasks with expressive millisecond-level precision.

For representation learning, several methods have been developed to produce symbolic music embeddings, useful as feature extractors for downstream classification tasks. These include MusicVAE [44], a variational autoencoder for capturing long-term structure; MusicBERT [4], which learns self-supervised representations via a bar-masking objective; and the CLaMP series of models [5,45,46], which employ contrastive learning techniques to build cross-modal representations with natural language descriptions.

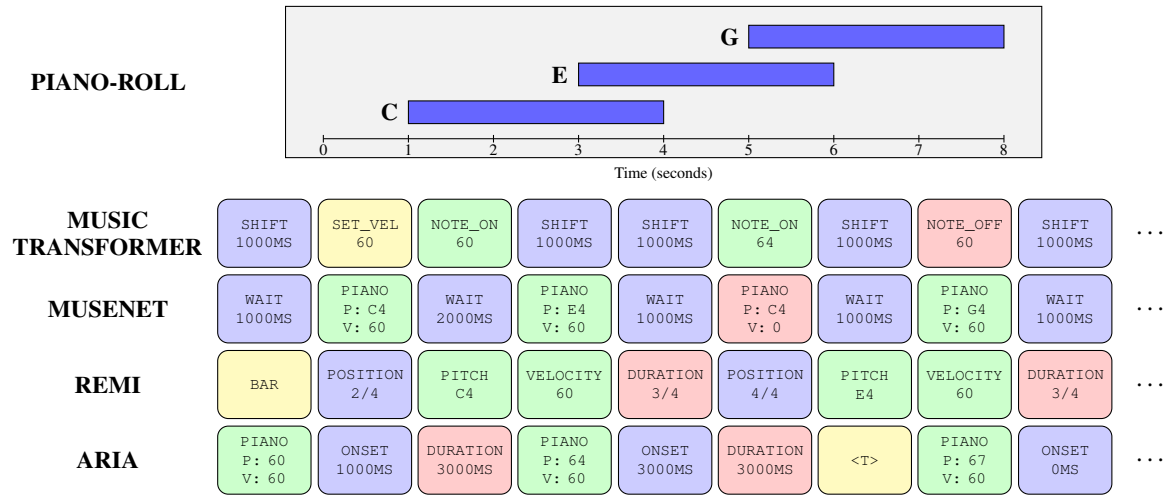


Figure 2. Comparison of different tokenizations of a piano-roll, using various approaches. Music Transformer [35] and MuseNet [38] track the passage of time using time-shift tokens, whereas Aria uses absolute onsets relative to the current segment. The REMI tokenizer [39] uses a neural beat-tracking model to estimate positions of notes and bar delimiters [47].

3. METHOD

To explore the capabilities of large-scale self-supervised models for piano performance, we first pretrained an autoregressive transformer model using next-token prediction on a refined subset of the Aria-MIDI dataset. We adopt this setup due to its versatility: next-token prediction has a proven track record in generative modeling for both symbolic and audio-based music [13,35], as well as adaptability to downstream tasks via finetuning [48]. Apart from the tokenization scheme, which we hand-designed, we used a conventional modern transformer architecture with minimal modifications, providing a standardized foundation for evaluating our hypothesis and supporting further research.

3.1 MIDI Tokenization

To autoregressively model MIDI files as streams of discrete tokens, we chose to use a temporal resolution of 10 milliseconds for note onsets and durations, and discretize note velocity values into 12 bins. Our tokenizer is designed to natively handle multi-track (multi-instrument) MIDI files by condensing the 128 MIDI instruments, corresponding to `program_change` MIDI messages, into 13 instrument classes, including one for percussion.

Given a MIDI file, we resolve its constituent `note_on` and `note_off` events into a list of notes. For non-percussion instruments, we tokenize a note with pitch p, velocity v, and absolute onset/offset in milliseconds $(t_{\text{on}}, t_{\text{off}})$ as a triple of tokens:

$$[\text{instrument}, p, v], [\text{onset: } t_{\text{on}}], [\text{duration: } t_{\text{off}} - t_{\text{on}}]$$

For percussion, we tokenize a note with note number n and onset t_{on} as:

$$[\text{drum}, n], [\text{onset: } t_{\text{on}}]$$

The tokenization of an entire MIDI file is constructed by concatenating the tokenizations of the constituent notes in order of onset. MIDI metadata, such as key, tempo, and time signature, is discarded, and other relevant musical information, such as the sustain pedal, is incorporated directly into the duration tokens. This schema is set apart from some popular tokenization techniques used for symbolic music, such as REMI [39] and text-based score representations ABC [43] and MusicXML [49], as it does not include beat or bar information, instead representing onsets and durations in milliseconds.

In the MIDI standard [50], `note_on` and `note_off` events are spaced temporally by specifying a number of ticks to wait before processing the next event. For Music Transformer and MuseNet, the authors incorporate this into their chosen MIDI tokenization schemes [35, 38], using *time-shift* tokens to separate notes rather than specifying their absolute onset times. However, emerging work has provided evidence that using time-shift tokens in this way may be suboptimal in transformer-based models, resulting in reduced accuracy in sequence-to-sequence piano transcription [51], and unstable rhythm or drifting bar lines in musical generations [39]. One possible explanation is that when using *relative-timing* tokenization, autoregressive models struggle to maintain an exact temporal representation of the prior context, as they must sum up many sequential time-shift values to calculate temporal relationships between notes with medium or long-term dependencies. Previous studies on large language models have demonstrated that transformers can struggle with exactly this sort of arithmetic [52, 53].

In preliminary investigations, we also observed negative effects when using relative-timing tokenizations, particularly on temporal instability in passages with rapid note sequences. To address these issues, we chose to adopt *absolute onset times* in our tokenizer. We implemented this by dividing the music into 5000-millisecond segments and recording note onsets relative to the start of each segment – this helped us avoid expanding the tokenizer's vocabulary

to include all possible absolute onset times. To remove ambiguity, we marked the start of each new segment using a special token: <T>. We designed this to resemble note timing using beat-position within a bar, however, unlike tokenization schemes that do this directly [39, 54], our approach is applicable to MIDI files that lack beat and bar information, such as those transcribed from solo piano recordings. Figure 2 illustrates how our approach differs from other approaches.

$$T_i - T_j = \begin{cases} \sum_{k=j+1}^{i} w_k & \text{Relative} \\ C(\texttt{<T>}, i, j) + \tilde{o}_i - \tilde{o}_j & \text{Hybrid (Ours)} \\ o_i - o_j & \text{Absolute} \end{cases} \quad (1)$$

Equation 1 demonstrates the arithmetic required to calculate the time separating two notes n_i and n_j across the different tokenization approaches, where w_k denotes the length of the time-shift message preceding note k, $C(\texttt{<T>}, i, j)$ represents the total time spanned by complete 5000ms segments between notes n_i and n_j, calculated by counting the number of segment tokens and multiplying by the segment duration, o_k represents the absolute onset time of note k, and $\tilde{o}_k$ represents the adjusted absolute onset time of note k relative to its 5000ms segment.

3.2 Model

Our model architecture builds upon the LLaMa 3.2 model family, chosen due to its effectiveness in autoregressive tasks across modalities [55]. Using the 1B parameter configuration as a starting point, we made several architectural modifications. Firstly, guided by established principles on model-data ratios for language models [56], we reduced the hidden state dimension (d_{model}) from 2048 to 1536. This decreased the parameter count by roughly half, balancing model capacity with computational efficiency for our dataset scale. Secondly, we simplified the architecture by opting for standard multi-head attention (with 24 heads) and layer normalization [57, 58], instead of grouped-query attention and RMS normalization as used in standard LLaMa 3 variants [59, 60].

Pretraining dataset. As our training corpus consists of automatically transcribed internet-sourced piano recordings, significant variability exists in transcription quality and content suitability, potentially introducing harmful biases or noisy data into downstream models. To mitigate this, we implemented rigorous preprocessing steps. To reduce memorization, we addressed extreme cases of composition duplication, such as repeated performances of overrepresented works, by applying filtering based on compositional metadata. Specifically, for composers with more than 250 instances of files containing opus and/or piece number tags, we retained at most 10 instances per opus/piece-number pair. For these same composers, we also discarded all other files that lack compositional identifiers. Additionally, we employed heuristic-based filtering, considering note density, pitch and duration entropy, silence, and indicators of repetitive content, to exclude problematic entries (e.g., *Black MIDI* [3]). Following these steps, our refined pretraining corpus comprises 820,944 MIDI files, amounting to 60,473 hours of solo piano music.

Pretraining recipe. We pretrained our model using standard next-token prediction on concatenated sequences of tokenized MIDI files, as detailed in Section 3.1. A sequence length of 8192 tokens was chosen to balance computational constraints with the need to learn meaningful short- and long-term dependencies within piano music. To enhance generalization and prevent overfitting, we utilized online data augmentation, randomly transposing ($\pm$5 semitones), varying tempo ($\pm$20%), and adjusting MIDI velocity ($\pm$10).

Generative finetuning. We produced a model variant tailored for generative piano-continuation tasks by applying a single-epoch finetuning phase after pretraining, annealing the learning rate to zero while training on higher-quality data. To enhance data quality, we removed all identified compositional duplicates, tightened existing quality filters, and introduced an additional filter aimed at excluding transcriptions of synthesized MIDI files [4]. Additionally, during this phase, each training sequence begins at the start of a new file (i.e., non-concatenated), and we insert a special token (<D>) approximately 100 tokens before the end of each training example to enable explicit inference-time control over generation endings.

3.3 Contrastive Representation Learning

To investigate the strength of the pretrained representations, we propose a secondary finetuning stage, adapting the pretrained model to generate *embeddings* of tokenized sequences. Our approach leverages the SimCLR framework for contrastive representation learning [61]. In SimCLR, an encoder is trained to produce similar embeddings for different *views* of the same training example while simultaneously pushing embeddings from unrelated examples apart through minimization of a contrastive loss. This approach has demonstrated strong results in music, capturing semantic relationships within embeddings effectively [62, 63], and has recently been combined with large pretrained language models to produce rich textual embeddings [64, 65].

To generate two distinct views of a MIDI file, we randomly extract two different contiguous slices, each comprising between 100 and 650 notes (approximately 300–2000 tokens). Each slice undergoes independent data augmentation using our standard procedures before tokenization. To produce sequence embeddings, we replace the original language modeling head with an embedding head, projecting the final hidden state into a 512-dimensional embedding space. We derive a slice's embedding from the hidden state associated with an end-of-sequence token appended after the final note token-triple.

To calculate the contrastive loss, we use the normalized temperature-scaled cross-entropy loss, *NT-Xent*, over minibatches of related embedding pairs:

$$\ell_{i,j} = -\log \frac{\exp\left(\text{sim}\left(z_i, z_j\right)/\tau\right)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp\left(\text{sim}\left(z_i, z_k\right)/\tau\right)} \quad (2)$$

[3] https://en.wikipedia.org/wiki/Black_MIDI

[4] Preprocessing details: https://github.com/loubbrad/aria-midi

Here, $\text{sim}(z_k, z_l)$ denotes the cosine similarity between normalized embeddings z_k and z_l, $\mathbb{1}_{[k \neq i]} \in \{0, 1\}$ is an indicator function, and τ is the temperature parameter. Each minibatch consists of N MIDI files, from which we construct N pairs of related embeddings (i.e., $2N$ total embeddings), $\{z_i, z_{i+N}\}_{i=1,\ldots,N}$, where both z_i and z_{i+N} are derived from two augmented views of the same file. We train the model by minimizing the symmetric loss: $L := \frac{1}{2}\sum_{k=1}^{N}(\ell_{k,k+N} + \ell_{k+N,k})$.

This setup has two key advantages. First, by extracting non-overlapping slices from the same file, the model learns embeddings reflecting higher-level musical semantics such as genre, composer, style, and performance nuances, rather than local details. This is important for musical performances, where standard supervised representation learning approaches, e.g., MuLan [66], are limited due to the descriptive subtlety and complexity of musical attributes. Second, our approach facilitates studying how effectively next-token prediction representations transfer to contrastive embedding frameworks. When trained from scratch, SimCLR-inspired training methods typically require large amounts of in-batch negatives, which pose significant VRAM constraints [61]. However, recent work on text embeddings shows that initializing contrastive training from pretrained models can alleviate this [64]. Thus, our method introduces a general-purpose semi-supervised framework for representation learning of symbolic music, which allows us to evaluate the transferability of next-token musical representations.

4. EXPERIMENTS

Having outlined our methodology, we evaluate the generative capabilities of our model, as well as the contrastive representation learning framework, in the context of piano performance. To understand its capabilities in the wider area of models for generative music and MIR, we compare our approach to both symbolic and audio-based baselines, utilizing Pianoteq [67] to synthesize MIDI files into audio.

4.1 Setup

We pretrained our model using the AdamW optimizer for 75 epochs over the training corpus. We used a learning rate of $3e$-4 with 1000 warmup steps, followed by a linear decay to 10% of the initial rate over the course of training. The model has approximately 650 million parameters and was pretrained for 9 days on 8 H100 GPUs with a batch size of 16 per GPU.

In the contrastive finetuning stage, we used a learning rate of $1e$-5 with the same linear decay schedule. We set the NT-Xent temperature parameter to $\tau = 0.1$. This phase lasted 25 epochs, during which each MIDI file contributes exactly one pair of augmented views per epoch. We trained on the reduced finetuning dataset described in Section 3.2; however, we relaxed the preprocessing constraints on compositional duplicates to encourage the model to distinguish between different performances of popular compositions.

Generative modeling. Following the generative finetuning procedure described in Section 3.2, we explore the generative capabilities of the resulting model by analyzing the *musical coherence* of continuations of short solo piano prompts. This methodology aligns with evaluations in previous work [13, 29], and mitigates taste bias by having participants evaluate continuations within the same musical style.

Compared Model	Wins	Ties	Losses	p-value
AM Transformer	38	0	6	$9.43e$-7
Suno 3.5	18	9	21	$7.49e$-1
MusicGen	49	1	0	$3.55e$-15
Ground Truth	15	9	17	$8.60e$-1

Table 1. Pairwise human preference results comparing musical coherence of 45-second continuations of 15-second prompts. We report the number of times our model won, tied, or lost against the listed model. P-values are computed using a two-sided binomial test on non-tied comparisons.

In our listening test, we asked 46 participants with at least one year of musical training to compare 45-second continuations generated from 15-second solo piano prompts, evaluating their musical coherence. Participants were presented with a series of random pairwise A/B comparisons, where they were asked to indicate their preferred continuation, guided by criteria such as melodic development, rhythmic structure, harmonic progression, and stylistic coherence. To generate test samples, we selected five prompts representing different subgenres of solo piano music, and generated eight continuations per prompt (totaling 40 continuations per model). We compared our model's outputs against several baselines, including Anticipatory Music Transformer (`music-large-800k`) [29], the audio-based generative models MusicGen (`large`) [16] and Suno 3.5 [30], and the human-composed ground-truth.

Contrastive embeddings. We evaluate our approach to learning contrastive embeddings by training linear classifiers on the frozen embeddings produced by different models and comparing their performance on held-out test sets. We assess performance using established benchmarks, Pianist8 [68] and VG-MIDI [69], as well as new benchmarks we derive from Aria-MIDI metadata. Specifically, we extracted label-balanced train-test splits comprising 10,000 and 1,000 files, respectively, for four classification tasks: Genre (2 classes), Musical Period (4 classes), Form (6 classes), and Composer (10 classes). For comparison, we include results from CLaMP 3 (`saas`) [46], M3 [45], and the audio-based model MERT [70]. Linear classifiers were trained on global file embeddings obtained by averaging slice embeddings within each file. We trained with a learning rate of $3e$-4 and a linear decay schedule to 0, running separate experiments with 10, 20, and 50 epochs, and reporting the best result.

Supervised finetuning. To complement our linear probe experiments, we evaluate how well our pretrained model adapts to supervised musical classification tasks, employing finetuning techniques inspired by NLP literature [48, 71]. For classifier finetuning, we replaced the language modeling head with a classification head, predicting labels directly from the hidden state of the end-of-sequence token. During

Model	Genre		Form		Musical Period		Composer		Pianist8		VG-MIDI	
	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1
Main Results												
MERT	83.00	83.00	63.89	63.90	69.50	68.94	69.60	69.30	65.06	65.18	45.45	40.37
M3	85.10	85.10	69.88	70.12	71.20	70.81	71.90	71.72	81.93	81.48	54.55	46.13
CLaMP 3	89.10	89.10	77.79	77.97	80.60	80.20	84.50	84.46	80.72	79.76	45.45	36.53
Aria_{Emb}	92.40	92.40	82.45	82.57	84.70	84.69	90.50	90.49	**91.57**	**92.38**	63.64	63.96
Aria_{Ft}	**93.20**	**93.20**	**87.53**	**87.59**	**86.50**	**86.53**	**96.30**	**96.32**	91.56	92.03	**68.18**	**69.55**
Embeddings												
$\text{Aria}^{\dagger}_{e=25}$	82.30	82.30	66.94	66.96	69.00	68.50	65.50	65.41	84.34	84.56	59.09	54.29
$\text{Aria}_{e=1}$	92.90	92.90	80.53	80.69	83.80	83.71	87.60	87.62	92.77	93.71	59.09	57.80
$\text{Aria}_{\tau=0.05}$	92.40	92.40	81.34	81.48	84.00	83.85	89.90	89.90	95.18	95.71	59.09	54.32
$\text{Aria}_{\tau=0.5}$	92.30	92.30	73.43	73.63	80.70	80.56	70.20	70.05	91.57	92.70	54.55	45.00
Finetuning												
$\text{Aria}_{n=100}$	89.50	89.50	68.26	68.20	70.20	70.64	65.30	64.10	-	-	-	-
$\text{Aria}_{n=200}$	91.10	91.10	75.25	75.54	75.10	75.68	78.10	78.08	-	-	-	-
$\text{Aria}_{n=500}$	90.80	90.80	79.31	79.49	80.90	80.91	85.20	85.18	-	-	-	-
$\text{Aria}_{n=1000}$	91.40	91.40	80.63	80.68	82.90	83.01	90.10	90.12	-	-	-	-

Table 2. Classification performance across symbolic music tasks. We report maximum accuracy (Acc) and macro-F1 scores (F1) for each task. *Main Results* compare our embedding model (Aria_{Emb}) and supervised finetuned model (Aria_{Ft}) to other models (MERT, M3, CLaMP 3). *Embedding* ablations vary key components of the contrastive learning setup: training epochs (e), temperature parameter (τ), and without pretraining (†), while keeping all other settings the same as Aria_{Emb}. *Finetuning* ablations show test-set performance as a function of the number of labeled training files (n).

this phase, we finetuned all model weights end-to-end using a learning rate of $1e$-5 (without warmup) with linear decay schedule, and applied dropout to residual connections, increasing the dropout rate linearly from $p_d = 0.0$ (first layer) to $p_d = 0.2$ (final layer). By systematically varying the number of labeled training examples, using class-balanced subsets, we analyze our pretrained model's ability to adapt to supervised symbolic MIR tasks in scenarios with limited labeled data. In each case, we trained for 10 epochs and report the results from the best-performing epoch.

4.2 Results

Table 1 reports the results of our listening test. Participants consistently preferred the musical coherence of continuations produced by our model over those from Anticipatory Music Transformer and MusicGen. This signals a notable improvement in symbolic models for piano performance generation, which we primarily attribute to the scale of our training dataset, given our standardized setup. It also highlights limitations in audio models like MusicGen, whose restricted context window necessitates sliding-window inference, diminishing coherence in longer generations. Conversely, we found no statistically significant preference difference between our model's outputs and either Suno 3.5 or human-composed ground-truth continuations. We acknowledge two key limitations: Firstly, we could not include closed-access models like AudioLM [13], despite their promising reported results on similar piano-continuation benchmarks. Secondly, our evaluation excludes popular symbolic models such as MuPT [3], as their bar-level timing representation (e.g., ABC notation) is incompatible with expressive millisecond-level MIDI performances.

Table 2 summarizes the results of our linear probe and supervised finetuning classification experiments, alongside an ablation study of training configurations for contrastive learning. Our proposed method for semi-supervised representation learning substantially improves results on all benchmarks, producing embeddings that capture diverse file-level musical attributes without incorporating metadata during training. The ablation study further highlights the importance of initializing contrastive training from pretrained next-token representations, demonstrating that our contrastive method is competitive only when applied as a finetuning stage. Notably, finetuning on one embedding pair per file for a single epoch ($\text{Aria}_{e=1}$) surpasses training from scratch on 25 pairs per file ($\text{Aria}^{\dagger}_{e=25}$). While this represents an advancement, we note that our benchmarks focus exclusively on piano performances, whereas the comparison models support multi-instrument MIDI or audio files. Finally, our supervised finetuning experiments demonstrate the strong adaptability of next-token prediction SSL frameworks to supervised symbolic MIR tasks. Our finetuned models achieve state-of-the-art classification performance on large datasets and perform surprisingly well on complex tasks, even when trained on limited labeled data.

5. CONCLUSION

We introduce Aria, an autoregressive generative transformer model designed to investigate the scalability of self-supervised learning for symbolic music modeling. Our experiments show that this pretraining framework effectively adapts to generative modeling, MIDI-embedding generation, and supervised MIR tasks. Moreover, our findings suggest that careful data curation and large-scale training can unlock new opportunities for downstream symbolic music applications, particularly in settings where data is scarce.

6. ACKNOWLEDGMENTS

This work was supported by UKRI and EPSRC under grant EP/S022694/1. Additional support was provided by EleutherAI and StabilityAI, as well as a compute grant from the Ministry of Science and ICT of Korea and Gwangju Metropolitan City.

7. REFERENCES

[1] J. Gui, T. Chen, J. Zhang, Q. Cao, Z. Sun, H. Luo, and D. Tao, "A survey on self-supervised learning: Algorithms, applications, and future trends," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.

[2] Y. Wang, S. Wu, J. Hu, X. Du, Y. Peng, Y. Huang, S. Fan, X. Li, F. Yu, and M. Sun, "Notagen: Advancing musicality in symbolic music generation with large language model training paradigms," *arXiv preprint arXiv:2502.18008*, 2025.

[3] X. Qu, Y. Bai, Y. Ma, Z. Zhou, K. M. Lo, J. Liu, R. Yuan, L. Min, X. Liu, T. Zhang *et al.*, "Mupt: A generative symbolic music pretrained transformer," *arXiv preprint arXiv:2404.06393*, 2024.

[4] M. Zeng, X. Tan, R. Wang, Z. Ju, T. Qin, and T.-Y. Liu, "Musicbert: Symbolic music understanding with large-scale pre-training," *arXiv preprint arXiv:2106.05630*, 2021.

[5] S. Wu, D. Yu, X. Tan, and M. Sun, "Clamp: Contrastive language-music pre-training for cross-modal symbolic music information retrieval," *arXiv preprint arXiv:2304.11029*, 2023.

[6] C. Raffel, "Learning-based methods for comparing sequences, with applications to audio-to-midi alignment and matching," Ph.D. dissertation, Columbia University, 2016.

[7] IMSLP. (2006) IMSLP/Petrucci music library. IMSLP. [Online]. Available: https://imslp.org

[8] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark *et al.*, "Learning transferable visual models from natural language supervision," in *International conference on machine learning*. PmLR, 2021, pp. 8748–8763.

[9] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[10] C. Zhou, P. Liu, P. Xu, S. Iyer, J. Sun, Y. Mao, X. Ma, A. Efrat, P. Yu, L. Yu *et al.*, "Lima: Less is more for alignment," *Advances in Neural Information Processing Systems*, vol. 36, pp. 55 006–55 021, 2023.

[11] A. Kolesnikov, L. Beyer, X. Zhai, J. Puigcerver, J. Yung, S. Gelly, and N. Houlsby, "Big transfer (bit): General visual representation learning," in *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part V 16*. Springer, 2020, pp. 491–507.

[12] F. Kreuk, G. Synnaeve, A. Polyak, U. Singer, A. Défossez, J. Copet, D. Parikh, Y. Taigman, and Y. Adi, "Audiogen: Textually guided audio generation," *arXiv preprint arXiv:2209.15352*, 2022.

[13] Z. Borsos, R. Marinier, D. Vincent, E. Kharitonov, O. Pietquin, M. Sharifi, D. Roblek, O. Teboul, D. Grangier, M. Tagliasacchi *et al.*, "Audiolm: A language modeling approach to audio generation," *IEEE/ACM transactions on audio, speech, and language processing*, vol. 31, pp. 2523–2533, 2023.

[14] A. Défossez, J. Copet, G. Synnaeve, and Y. Adi, "High fidelity neural audio compression," *arXiv preprint arXiv:2210.13438*, 2022.

[15] N. Zeghidour, A. Luebs, A. Omran, J. Skoglund, and M. Tagliasacchi, "Soundstream: An end-to-end neural audio codec," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 30, pp. 495–507, 2021.

[16] A. Agostinelli, T. I. Denk, Z. Borsos, J. Engel, M. Verzetti, A. Caillon, Q. Huang, A. Jansen, A. Roberts, M. Tagliasacchi *et al.*, "Musiclm: Generating music from text," *arXiv preprint arXiv:2301.11325*, 2023.

[17] J. Copet, F. Kreuk, I. Gat, T. Remez, D. Kant, G. Synnaeve, Y. Adi, and A. Défossez, "Simple and controllable music generation," *Advances in Neural Information Processing Systems*, vol. 36, pp. 47 704–47 720, 2023.

[18] Z. Borsos, M. Sharifi, D. Vincent, E. Kharitonov, N. Zeghidour, and M. Tagliasacchi, "Soundstorm: Efficient parallel audio generation," *arXiv preprint arXiv:2305.09636*, 2023.

[19] W.-N. Hsu, B. Bolte, Y.-H. H. Tsai, K. Lakhotia, R. Salakhutdinov, and A. Mohamed, "Hubert: Self-supervised speech representation learning by masked prediction of hidden units," 2021. [Online]. Available: https://arxiv.org/abs/2106.07447

[20] A. Baevski, Y. Zhou, A. Mohamed, and M. Auli, "wav2vec 2.0: A framework for self-supervised learning of speech representations," *Advances in neural information processing systems*, vol. 33, pp. 12 449–12 460, 2020.

[21] E. Benetos, S. Dixon, Z. Duan, and S. Ewert, "Automatic music transcription: An overview," *IEEE Signal Processing Magazine*, vol. 36, no. 1, pp. 20–30, 2018.

[22] Q. Kong, B. Li, X. Song, Y. Wan, and Y. Wang, "High-resolution piano transcription with pedals by regressing onset and offset times," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 29, pp. 3707–3717, 2021.

[23] K. Toyama, T. Akama, Y. Ikemiya, Y. Takida, W.-H. Liao, and Y. Mitsufuji, "Automatic piano transcription with hierarchical frequency-time transformer," *arXiv preprint arXiv:2307.04305*, 2023.

[24] Y. Yan and Z. Duan, "Scoring time intervals using non-hierarchical transformer for automatic piano transcription," *arXiv preprint arXiv:2404.09466*, 2024.

[25] Q. Kong, B. Li, J. Chen, and Y. Wang, "Giantmidi-piano: A large-scale midi dataset for classical piano music," *arXiv preprint arXiv:2010.07061*, 2020.

[26] H. Zhang, J. Tang, S. Rafee, S. Dixon, and G. Fazekas, "Atepp: A dataset of automatically transcribed expressive piano performance," in *Proceedings of the 23rd International Society for Music Information Retrieval Conference (ISMIR)*, 2022.

[27] D. Edwards, S. Dixon, and E. Benetos, "Pijama: Piano jazz with automatic midi annotations," *Transactions of the International Society for Music Information Retrieval*, 2023.

[28] L. Bradshaw and S. Colton, "Aria-midi: A dataset of piano midi files for symbolic music modeling," in *International Conference on Learning Representations*, 2025. [Online]. Available: https://openreview.net/forum?id=X5hrhgndxW

[29] J. Thickstun, D. Hall, C. Donahue, and P. Liang, "Anticipatory music transformer," *arXiv preprint arXiv:2306.08620*, 2023.

[30] I. Suno, "Suno AI v3.5," 2024, computer software. [Online]. Available: https://sunnoai.com/v3-5/

[31] G. Hadjeres, F. Pachet, and F. Nielsen, "Deepbach: A steerable model for bach chorales generation," in *International conference on machine learning*. PMLR, 2017, pp. 1362–1371.

[32] C.-Z. A. Huang, T. Cooijmans, A. Roberts, A. Courville, and D. Eck, "Counterpoint by convolution," *arXiv preprint arXiv:1903.07227*, 2019.

[33] F. T. Liang, M. Gotham, M. Johnson, and J. Shotton, "Automatic stylistic composition of bach chorales with deep lstm," in *ISMIR*, 2017, pp. 449–456.

[34] S. Oore, I. Simon, S. Dieleman, D. Eck, and K. Simonyan, "This time with feeling: Learning expressive musical performance," *Neural Computing and Applications*, vol. 32, pp. 955–967, 2020.

[35] C.-Z. A. Huang, A. Vaswani, J. Uszkoreit, N. Shazeer, I. Simon, C. Hawthorne, A. M. Dai, M. D. Hoffman, M. Dinculescu, and D. Eck, "Music transformer," *arXiv preprint arXiv:1809.04281*, 2018.

[36] C. Hawthorne, A. Stasyuk, A. Roberts, I. Simon, C.-Z. A. Huang, S. Dieleman, E. Elsen, J. Engel, and D. Eck, "Enabling factorized piano music modeling and generation with the maestro dataset," *arXiv preprint arXiv:1810.12247*, 2018.

[37] I. Simon, C.-Z. A. Huang, J. Engel, C. Hawthorne, and M. Dinculescu, "Generating piano music with transformer," https://magenta.tensorflow.org/piano-transformer, September 2019, blog post. [Online]. Available: https://magenta.tensorflow.org/piano-transformer

[38] C. Payne, "Musenet," 2019, openAI, 25 Apr. 2019. [Online]. Available: https://openai.com/blog/musenet

[39] Y.-S. Huang and Y.-H. Yang, "Pop music transformer: Beat-based modeling and generation of expressive pop piano compositions," in *Proceedings of the 28th ACM international conference on multimedia*, 2020, pp. 1180–1188.

[40] B. Yu, P. Lu, R. Wang, W. Hu, X. Tan, W. Ye, S. Zhang, T. Qin, and T.-Y. Liu, "Museformer: Transformer with fine-and coarse-grained attention for music generation," *Advances in neural information processing systems*, vol. 35, pp. 1376–1388, 2022.

[41] D. von Rütte, L. Biggio, Y. Kilcher, and T. Hofmann, "Figaro: Generating symbolic music with fine-grained artistic control," *arXiv preprint arXiv:2201.10936*, 2022.

[42] P. Lu, X. Xu, C. Kang, B. Yu, C. Xing, X. Tan, and J. Bian, "Musecoco: Generating symbolic music from text," *arXiv preprint arXiv:2306.00110*, 2023.

[43] C. Walshaw, "Abc notation," abcnotation.com, 2008, retrieved 1 March 2008.

[44] A. Roberts, J. Engel, C. Raffel, C. Hawthorne, and D. Eck, "A hierarchical latent vector model for learning long-term structure in music," in *International conference on machine learning*. PMLR, 2018, pp. 4364–4373.

[45] S. Wu, Y. Wang, R. Yuan, Z. Guo, X. Tan, G. Zhang, M. Zhou, J. Chen, X. Mu, Y. Gao *et al.*, "Clamp 2: Multimodal music information retrieval across 101 languages using large language models," *arXiv preprint arXiv:2410.13267*, 2024.

[46] S. Wu, Z. Guo, R. Yuan, J. Jiang, S. Doh, G. Xia, J. Nam, X. Li, F. Yu, and M. Sun, "Clamp 3: Universal music information retrieval across unaligned modalities and unseen languages," *arXiv preprint arXiv:2502.10362*, 2025.

[47] S. Böck, F. Krebs, and G. Widmer, "Joint beat and downbeat tracking with recurrent neural networks." in *ISMIR*. New York City, 2016, pp. 255–261.

[48] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of machine learning research*, vol. 21, no. 140, pp. 1–67, 2020.

[49] M. Good, "MusicXML: An internet-friendly format for sheet music," in *Proceedings of XML 2001 Conference*, 2001. [Online]. Available: https://michaelgood.info/publications/music/musicxml-an-internet-friendly-format-for-sheet-music/

[50] "MIDI specification," 1996. [Online]. Available: https://midi.org/midi-1-0-detailed-specification

[51] C. Hawthorne, I. Simon, R. Swavely, E. Manilow, and J. Engel, "Sequence-to-sequence piano transcription with transformers," *arXiv preprint arXiv:2107.09142*, 2021.

[52] N. Lee, K. Sreenivasan, J. D. Lee, K. Lee, and D. Papailiopoulos, "Teaching arithmetic to small transformers," 2023. [Online]. Available: https://arxiv.org/abs/2307.03381

[53] S. McLeish, A. Bansal, A. Stein, N. Jain, J. Kirchenbauer, B. R. Bartoldson, B. Kailkhura, A. Bhatele, J. Geiping, A. Schwarzschild, and T. Goldstein, "Transformers can do arithmetic with the right embeddings," 2024. [Online]. Available: https://arxiv.org/abs/2405.17399

[54] W.-Y. Hsiao, J.-Y. Liu, Y.-C. Yeh, and Y.-H. Yang, "Compound word transformer: Learning to compose full-song music over dynamic directed hypergraphs," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 1, 2021, pp. 178–186.

[55] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[56] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. d. L. Casas, L. A. Hendricks, J. Welbl, A. Clark *et al.*, "Training compute-optimal large language models," *arXiv preprint arXiv:2203.15556*, 2022.

[57] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[58] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," *arXiv preprint arXiv:1607.06450*, 2016.

[59] J. Ainslie, J. Lee-Thorp, M. De Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "Gqa: Training generalized multi-query transformer models from multi-head checkpoints," *arXiv preprint arXiv:2305.13245*, 2023.

[60] B. Zhang and R. Sennrich, "Root mean square layer normalization," *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[61] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *International conference on machine learning*, 2020, pp. 1597–1607.

[62] J. Spijkervet and J. A. Burgoyne, "Contrastive learning of musical representations," *arXiv preprint arXiv:2103.09410*, 2021.

[63] J. Choi, S. Jang, H. Cho, and S. Chung, "Towards proper contrastive self-supervised learning strategies for music audio representation," in *2022 IEEE International Conference on Multimedia and Expo (ICME)*. IEEE, 2022, pp. 1–6.

[64] T. Gao, X. Yao, and D. Chen, "Simcse: Simple contrastive learning of sentence embeddings," *arXiv preprint arXiv:2104.08821*, 2021.

[65] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei, "Improving text embeddings with large language models," *arXiv preprint arXiv:2401.00368*, 2023.

[66] Q. Huang, A. Jansen, J. Lee, R. Ganti, J. Y. Li, and D. P. Ellis, "Mulan: A joint embedding of music audio and natural language," *arXiv preprint arXiv:2208.12415*, 2022.

[67] Modartt, "Pianoteq," https://www.modartt.com/pianoteq, accessed: 2025-03-28.

[68] Y.-H. Chou, I. Chen, C.-J. Chang, J. Ching, Y.-H. Yang *et al.*, "Midibert-piano: Large-scale pre-training for symbolic music understanding," *arXiv preprint arXiv:2107.05223*, vol. 2, 2021.

[69] L. N. Ferreira and J. Whitehead, "Learning to generate music with sentiment," *arXiv preprint arXiv:2103.06125*, 2021.

[70] Y. Li, R. Yuan, G. Zhang, Y. Ma, X. Chen, H. Yin, C. Xiao, C. Lin, A. Ragni, E. Benetos *et al.*, "Mert: Acoustic music understanding model with large-scale self-supervised training," *arXiv preprint arXiv:2306.00107*, 2023.

[71] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, 2019, pp. 4171–4186.